IBM Innovation Centre for Education

NCS NihonCommunicationSolutions

YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT

Project – High Level Design

on

End to End Encryption For Supply Chain Cloud App

Student Name: T Abdul Sameer

Register Number: 23BCCE135

Course: BCA in Cloud Computing, Cyber Security & Ethical Hacking with IBM

Industry Mentor: Mr. Shashank **Guided by:** Ms. Vijayalaxmi

Table of Contents

Sl.no	Contents	Page
1	**Introduction**	1-2
1.1	Scope of the document	
1.2	Intended Audience	
1.3	System overview	
2	**System Design**	3-9
2.1	Application Design	
2.2	Process Flow	
2.3	Information Flow	
2.4	Components Design	
2.5	Key Design Considerations	
2.6	API Catalogue	
3	**Data Design**	10-11
3.1	Data Model	
3.2	Data Access Mechanism	
3.3	Data Retention Policies	
3.4	Data Migration	
4	**Interfaces**	12
5	**State and Session Management**	13
6	**Caching**	14
7	**Non-Functional Requirements**	15-16
7.1	Security Aspects	
7.2	Performance Aspects	
8	**References**	17

1. Introduction

1. Scope of the document

This High Level Design (HLD) document describes the overall architecture, design approach, and working of the End to End Encryption for Supply Chain Cloud App. It explains how the system is structured to protect supply chain cloud data during storage, sharing, and transmission through a secure web-based application.

2. Intended Audience

This document is intended for project evaluators, developers, and students who want to understand the system design at a conceptual level. It provides a clear and structured overview of how the system operates without going into low-level implementation details. It is especially useful for individuals interested in cloud security, secure document sharing, encryption-based applications, and protected supply-chain communication. The document also serves as a reference for future enhancements and system scalability considerations.

3. System overview

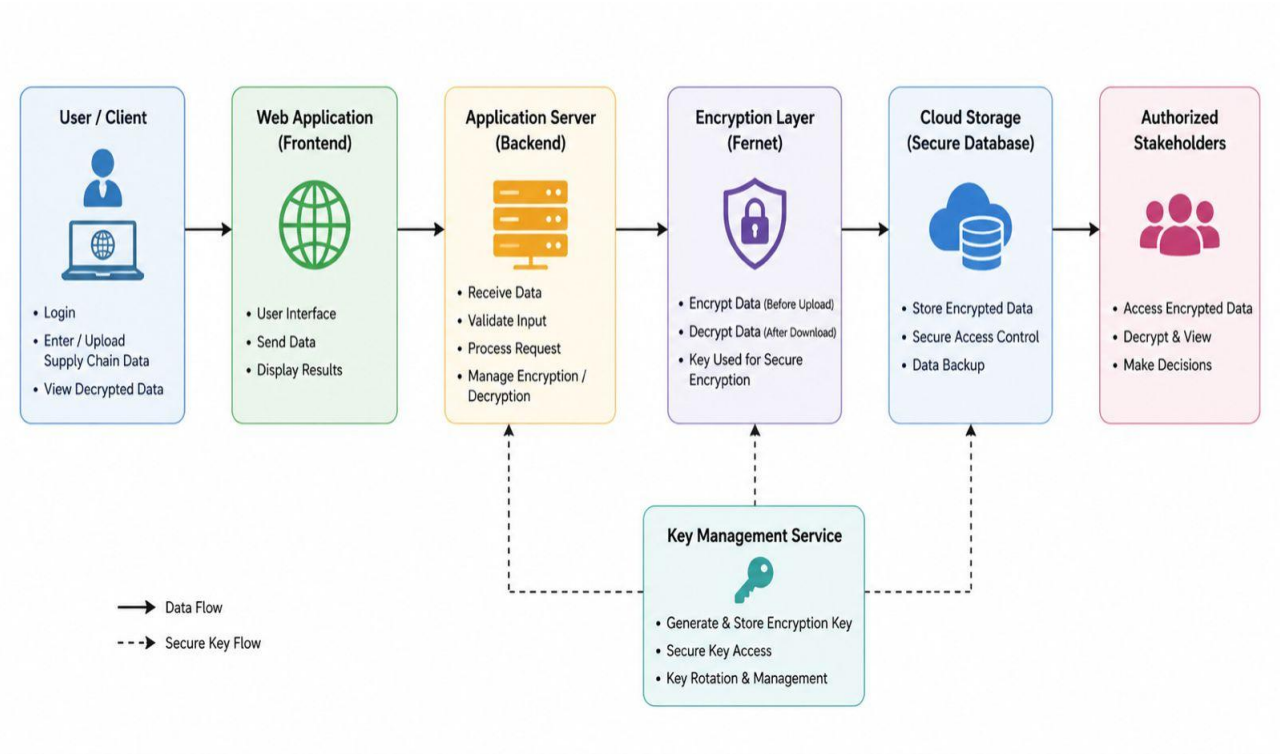
The system is designed as a secure cloud-based application for supply chain communication. It allows authorized users such as Admins, Manufacturers, Suppliers, Warehouses, and Distributors to upload, store, and access encrypted files or messages. Before data is stored or shared, it is converted into encrypted format using Fernet-based symmetric encryption. Only authorized users with the proper access flow can retrieve and decrypt the information. The system combines a Flask-based web interface, encrypted storage, access-controlled workflows, and audit visibility to support secure business communication.

2. System Design

2.1 Application Design

The application is designed as a simplified secure cloud platform. It consists of a Flask-based web application, encryption module, file storage layer, and role-based user access. The architecture follows a modular approach, separating user interaction, encryption, secure storage, access handling, and audit monitoring.

The frontend is developed using HTML and CSS to provide a simple dashboard, upload page, and file-access pages. The backend is developed using Python Flask, which handles user requests, routes, secure file processing, and system responses.

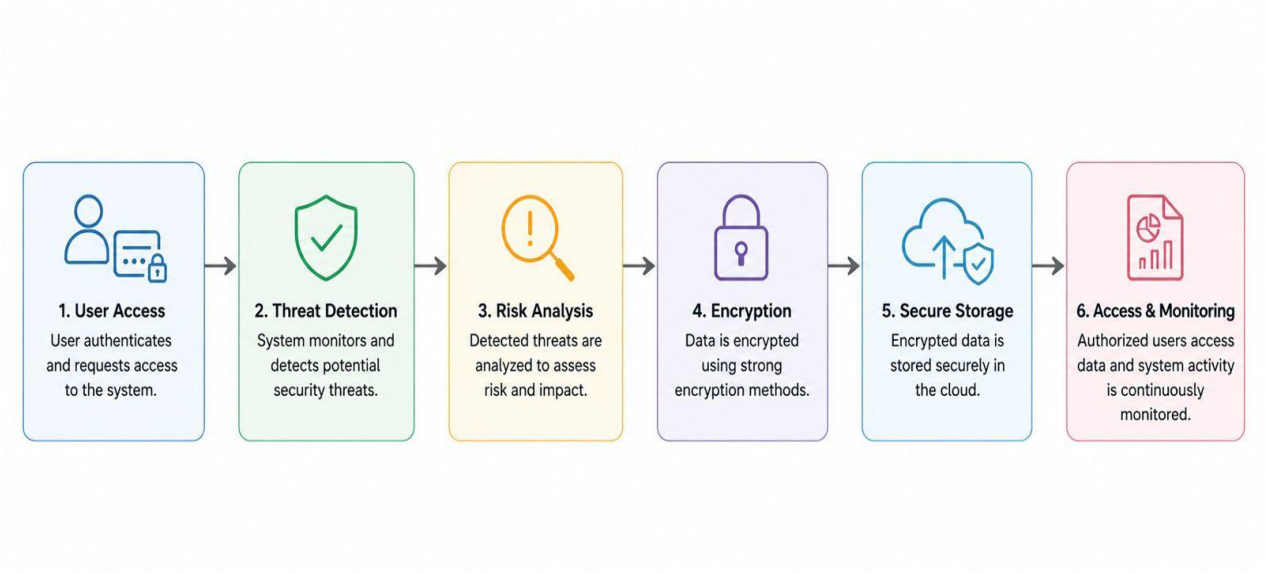


The encryption layer uses the Fernet algorithm to protect manufacturing data before it is stored in the application. The system is designed so that plain text business data is not directly stored in readable form.

2.2 Process Flow

The process begins when an authorized user logs into the application. The user then uploads a supply chain file, or enters sensitive information that must be protected. The encryption module generates encrypted content using the secret key and passes the ciphertext to the storage layer.

.



When another authorized user requests access, the system verifies the request, retrieves the encrypted content, and sends it to the decryption module. The original data is shown only after successful decryption and authorization. This complete process ensures confidentiality, controlled access, and secure handling of supply chain cloud data.

Finally, the original supply chain information is displayed only to the authorized user after successful decryption. This complete flow ensures confidentiality, secure storage, controlled access, and safe sharing of sensitive supply chain cloud data among trusted participants.

2.3 Information Flow

The information flow in the system begins when a user submits plain text or file-based manufacturing data through the web interface. This request is received by the Flask backend, which forwards the data to the encryption module.

Source	Destination	Data Type	Description
User Interface	Flask Backend	HTTP Requests & User Input	Users interact with the application, sending commands and supply chain data
Plain Text Supply Chain Data	Encryption Module	Sensitive Supply Chain Records	Raw data like item descriptions, supplier details, and inventory requiring protection before storage
Encryption Module	Encrypted Storage	Ciphertext	The module encrypts the plain text data before sending it for secure storage (e.g., Database)
Encrypted Storage	Decryption Module	Ciphertext	Retrieving encrypted data from storage when an authorized access request is made
Decryption Module	User Interface	Plain Text Supply Chain Data	The module decrypts the data and presents the human-readable information to authorized users
Flask Backend	Audit Log	Security Event Logs	All user actions, encryption/decryption events, and access are logged for security and compliance monitoring

2.5 Key Design Considerations

The system is designed to be simple, secure, and suitable for an academic cloud security project. The primary design goal is to ensure that supply chain cloud data remains protected from unauthorized access while keeping the implementation lightweight and understandable.

End-to-end style protection is an important design consideration. Sensitive data is encrypted before storage and is only converted back into readable form for authorized access. This helps maintain confidentiality during storage and sharing.

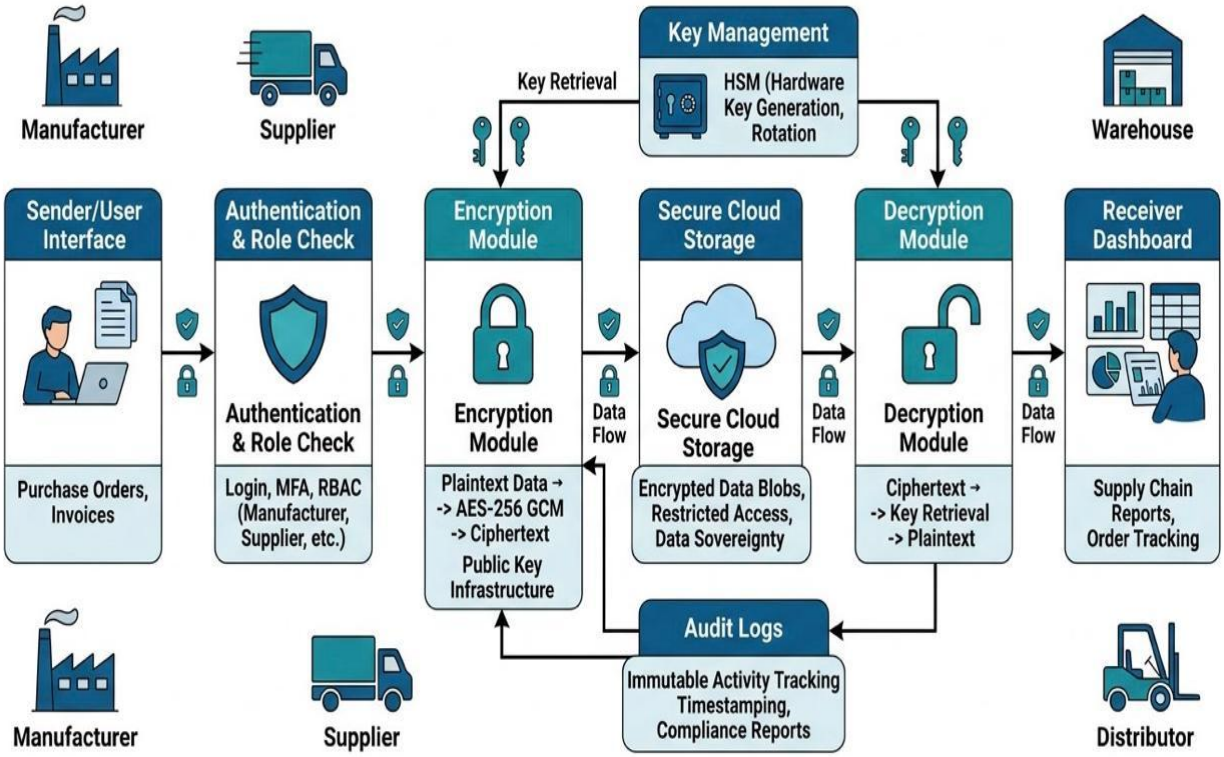
Another major consideration is role-based access control. Different users such as Admin, Manufacturer, Warehouse, and Distributor should only access the part of the system relevant to their responsibilities. This improves both security and usability.

The design also focuses on modularity. By separating the frontend, backend, encryption, decryption, storage, and logging components, the system becomes easier to maintain and improve. In future, the same design can be extended with cloud database integration, stronger key management, and multi-user enterprise deployment.

3. Data Design

1. Data Model

The system uses a simple secure data model to handle supply chain communication records. Each record may represent an uploaded file or message and includes important fields such as user identity, encrypted content, filename, timestamp, access role, and decryption eligibility.



2. Data Access Mechanism

The Flask application reads encrypted files or encrypted text records from the storage layer and displays them through the dashboard or files page. When a user requests access, the system checks authorization, fetches the encrypted data, and passes it through the decryption process before display.

This access mechanism ensures that users only see readable content after proper validation. It also keeps the implementation simple and suitable for educational use.

3. Data Retention Policies

The system maintains encrypted records and related audit entries for continuous monitoring and secure access. These records help in tracking uploads, decryption events, and user activity over time.

The stored data is not automatically deleted in the current version, which allows demonstration and review of project activity. In future improvements, retention rules such as periodic cleanup, archival, or expiry-based deletion can be introduced.

4. Data Migration

The current design uses simple local storage or file-based storage for ease of development and demonstration. This approach is suitable for a student project and small-scale testing.

For larger deployments, the system can be migrated to a database solution such as MySQL or MongoDB, along with secure cloud object storage. This would improve scalability, allow easier querying, and support larger volumes of encrypted records and user data.

4. Interfaces

The system provides a web-based interface using the Flask framework for secure file and message management. The main interface is the dashboard, which displays options such as upload, view files, decrypt data, and role-based navigation.

The upload interface allows authorized users to choose a file and submit it securely to the backend. The files interface displays available stored encrypted files, while the decryption interface shows readable output only after successful authorization and decryption.

The audit log interface, available to Admin users, displays important security events such as login attempts, uploads, decrypt actions, and failures. Overall, the interfaces are designed to be simple, user-friendly, and effective for secure supply chain cloud communication.

5. State and Session Management

The system follows a structured session-based approach for handling secure user activity. Initially, the application starts in an unauthenticated state where the user must log in with valid credentials.

After successful authentication, the session moves into an active authorized state. In this state, the system grants access according to the user role. The user may then upload encrypted files, request stored records, or perform role-specific actions.

When an encrypted file is requested, the application enters a secure access state, where the request is validated and the decryption workflow is triggered. The session then returns to the active monitoring state after the action is completed.

All important activities are recorded in the audit trail. On logout, the session is terminated and the user must reauthenticate to continue using the system.

6. Caching

The encryption system uses a lightweight caching strategy to improve performance and reduce repeated processing. Since users may access the same metadata or file list multiple times, the application can reuse directory listings, session-level user state, and recently loaded non-sensitive metadata.

The system avoids caching decrypted sensitive content for long periods. Instead, it focuses on temporary handling of authorized responses, recent file references, and page rendering support. This approach reduces unnecessary processing while still maintaining confidentiality.

Overall, caching in this system is kept minimal and controlled so that security is not weakened for the sake of speed.

7. Non-Functional Requirements

1. Security Aspects

The honeypot system is designed to be lightweight and efficient. It uses Python socket programming, which allows fast handling of incoming connection requests with minimal dela

Aspect	Description
Access Control	The system does not allow real interaction; all connections are immediately denied to prevent unauthorized access.
Data Protection	Only attacker-related data such as IP address, timestamp, and attempt count is stored; no sensitive system data is exposed.
Isolation	The honeypot acts as a decoy system, ensuring the real system remains protected from external threats.
Attack Detection	Repeated attempts from the same IP are tracked and marked as suspicious activity.
Logging Security	All connection attempts are securely recorded in the log file for monitoring and analysis.
Safe Response	The system sends a fake “Access Denied” response to prevent attackers from gaining real system information.

7. Non-Functional Requirements

1. Security Aspects

The honeypot system is designed to be lightweight and efficient. It uses Python socket programming, which allows fast handling of incoming connection requests with minimal dela

Aspect	Description
Access Control	Only authorized users can access system features based on role
Data Protection	Sensitive files and messages are stored in encrypted form
Confidentiality	Plain text business data is not directly stored in readable format .
Audit logging	Important actions are recorded for monitoring and accountability
Secure Retrieval	Only validated users can request decrypted output .
Role Isolation	Different users see only the functions assigned to their roles

7.2 Performance Aspects

The system provides a secure environment for monitoring unauthorized access attempts. It does not allow any real interaction with the system, as all incoming connections are immediately denied.

Aspect	Description
Fast Processing	Encryption and decryption operations complete quickly for normal academic-scale usage .
Lightweight Design	Uses Flask, local storage, and modular components for simplicity
Efficient Access	Encrypted files are retrieved and processed with minimal delay
Scalability	Design can be extended to database and cloud storage in future
System Stability	The application remains stable during repeated uploads and access requests
Maintainability	Modular structure makes debugging and extension easier

8. References

1. Python Documentation –

https://docs.python.org/3/

2. Flask Documentation –

https://flask.palletsprojects.com/

3. Cryptography Library (Fernet) Documentation -

https://cryptography.io/

4. OWASP Cryptographic Storage Guidance –

https://cheatsheetseries.owasp.org/

5. General Concepts of End-to-End Encryption

- https://en.wikipedia.org/wiki/End-to-end_encryption